Deliverable 1: design and an executable proposal path

Individual · 10% of course · Due October 9, 2026, 23:59 Toronto time

Read `00_Project_Overview` first. Its identifiers, duration, fixture and baseline policy are normative. This first release uses a supplied replay response; a live model is not required yet. Preserve this codebase for D2 and D3.

Required work

D1.1 — Domain model. Implement booking, space and proposal representations that protect the stated valid states. Validate identifiers and duration. Distinguish a proposed reassignment from a completed change. Demonstrate separate object instances and controlled state access. Avoid exposing a mutable internal collection that bypasses validation.

D1.2 — Design diagrams. Supply one class diagram showing your main domain and application responsibilities, visibility where relevant, important associations and multiplicities. Supply one sequence diagram for reading B1, evaluating B12 and creating a pending proposal. Label the approval boundary even though operator execution is not yet required. Use correct UML arrows; an interface is not a superclass. Diagrams may be drawn with a supported tool or legibly produced by another method, but must be readable in PDF.

D1.3 — Executable path. With the supplied fixture, feed the scripted proposed target B12 to your application. Retrieve B1, evaluate baseline eligibility, capture booking/policy version 0/1, and create a pending proposal with an application-generated identifier. Display a structured result. The booking must remain at A17, version 0. Do not approve or execute automatically. Repeat the exercise with D09 and an unknown target; reject them without changing a booking.

D1.4 — Tests. Include automated checks for duration 1 and 24, rejected 0 and 25, null/invalid identifiers, eligible B12, occupied D09, unknown target, and unchanged booking state after proposing or rejecting. Use at least one Jupiter test class; the required cases determine adequacy, not an arbitrary maximum. Record the actual test command and result. When a dependency-installation problem prevents JUnit, retain the test source, run the independent checks and report the blocker; the TA must resolve the supported environment.

D1.5 — Explanation. In at most four pages of `DESIGN.pdf`, excluding the two diagrams if they occupy separate pages, explain the responsibilities, one alternative you considered, two informal operation contracts and the most important current limitation. Refer to exact source types or methods. State which infrastructure you reused and which parts you implemented.

Observable acceptance cases

Case	Initial/input	Required observation
D1-A	`new Booking` with 1 or 24 hours	Constructed object preserves that duration
D1-B	0 or 25 hours	Clear rejection; no usable invalid booking
D1-C	B1, proposed B12, policy 1	Pending proposal; booking still A17/version 0
D1-D	B1, proposed D09	Rejection; occupancy and booking unchanged
D1-E	Unknown target	Controlled error; no proposal/booking mutation
D1-F	Externally obtained snapshot	Caller cannot mutate protected internal state

An implementation can use different internal names. Include `results/d1.txt` showing how the assessor runs these cases. No web UI, persistent database, approval engine or real model is required for D1.

Rubric: 100 release points

Criterion	Points	Full-credit expectation
Domain validity and encapsulation	20	Valid ranges and identities enforced; controlled access and clear responsibilities
Class and sequence diagrams	20	Correct notation, useful relationships, and consistency with submitted code
Executable proposal path	25	Required accepted/rejected cases work; no booking mutation before approval
Test design and genuine results	20	Required boundaries and negative cases, correct expected results, repeatable command
Design explanation and alternatives	10	Precise contracts, one justified alternative and stated limitations
Reproducibility and attribution	5	Complete ZIP, usable README, supplied-code and AI disclosure

A missing executable path loses the relevant implementation marks; diagrams alone cannot receive them. A test that only checks that no exception occurred is insufficient for the unchanged-state requirement. A correct implementation with a different justified decomposition can receive full marks.

Submission and individual authorship

This is an individual project. Submit your own repository snapshot and explain every included design decision. Do not submit a shared implementation, team report or another student's source. Discussion of general concepts is permitted; exchange of assessed code, diagrams or solutions is not. Designated AI assistance is permitted with the disclosure below. Supplied infrastructure is identified explicitly and may be retained with attribution.

Submit one ZIP through the named eClass assignment before **23:59 Toronto time** on the stated date. Name it `EECS3311_Dn_STUDENTNUMBER.zip`, replacing `Dn` and `STUDENTNUMBER`. Its single top-level folder must contain `src/`, `test/`, `run.sh`, `README.md`, `DESIGN.pdf`, `AI_USE.md`, `TEST_PLAN.md`, `CHANGELOG.md` and `results/`. Keep generated `.class` files, IDE caches, model weights, secrets and downloaded libraries out. A private Git repository is recommended for development; the submitted ZIP is the marking snapshot. Do not rely on a remote branch changing after the deadline. Keep a copy of the uploaded receipt and ZIP.

`README.md` must state the JDK, exact execution commands, supported scenario(s), known limitations and any permitted starter files reused. Both `bash run.sh check` and the appropriate prepared demonstration must run from the extracted project root. Jupiter tests use `bash setup-junit.sh` once on the supported network, then `bash run.sh test`; the TA will provide/cache the pinned dependency. Include genuine command output with date, command and environment in `results/`. Do not label a plain-Java check run as a JUnit run or label replay as a live-model run.

In `AI_USE.md`, identify each tool/model used, date or version when known, the task it helped with, the source files or diagrams affected, what you changed or rejected, and how you checked the result. State "No generative AI assistance used" when applicable. Whole personal chat histories are not required. Remove credentials and personal information from any excerpt. Code produced with assistance remains your responsibility. The instructor may ask you to explain or make a small change to your submitted work under the course's academic-integrity procedures; there is no separate undisclosed viva mark.

Submit even if one part is incomplete: identify the affected requirement precisely. Do not hide a failure by deleting the test or editing a captured result. Required extensions cannot be replaced by a polished user interface. Optional features earn no substitute marks for missing core requirements. Approved accommodations and extensions take precedence over ordinary deadlines; request assistance through eClass promptly. Late/absence arrangements are governed by the published syllabus and approved course procedures, not by this brief inventing a separate penalty.